# Assignment-19.2

## 2303A51304

## Batch-05

**Task-01:**

**Prompt:**

```
# file_read_write.py

# write data to file
with open("input.txt", "w") as f:
    f.write("Hello World\n")
    f.write("Python File Handling Example")

# read data from file
with open("input.txt", "r") as f:
    content = f.read()

print("File Content:")
print(content)
```

Convert the following Python file handling program into an equivalent Java program.

Requirements:

Perform the same file read and write operations.

Use proper Java file I/O classes.

Handle exceptions correctly.

Maintain the same console output and file content.

Provide complete runnable Java code with compile and run steps.

**Code:**

```
%%writefile FileReadWrite.java

import java.io.FileWriter;
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;

public class FileReadWrite {
    public static void main(String[] args) {
        // Define the file name
        String fileName = "input.txt";

        // Write data to file
        try (FileWriter writer = new FileWriter(fileName)) {
            writer.write("Hello World\n");
            writer.write("Python File Handling");
            System.out.println("Successfully wrote to the file.");
        } catch (IOException e) {
            System.err.println("Error writing to file: " + e.getMessage());
        }

        // Read data from file
        System.out.println("\nFile Content:");
        try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
```

```
            }
        } catch (IOException e) {
            System.err.println("Error reading from file: " + e.getMessage());
        }
    }
}
```

```
Overwriting FileReadWrite.java
```

## Compile and Run the Java Program

First, compile the Java code using `javac`:

```
!javac FileReadWrite.java
```

Then, run the compiled Java program using `java`:

```
!java FileReadWrite
```

```
Successfully wrote to the file.

File Content:
Hello World
Python File Handling
```

**Output:**

```
!java FileReadWrite

Successfully wrote to the file.

File Content:
Hello World
Python File Handling
```

**Explanation:**

The Python program creates a file named sample.txt, writes text into it, and then reads the same content from the file.
It uses write mode to store data and read mode to retrieve the stored information.
The Java program performs the same operations using BufferedWriter and BufferedReader classes for file handling.
Exception handling and try-with-resources ensure files are safely opened and closed in Java.
Both programs display the file content on the console, producing identical output.

**Task-02:**

**Prompt:**

Convert the following C++ sorting program into an equivalent Python program.

Requirements:

Maintain the same sorting logic and behavior.

Ensure correct function conversion from C++ to Python.

Produce identical sorted output.

Provide complete runnable Python code.

Include sample test cases to verify correctness.

C++ Code:

```
// bubble_sort.cpp
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {5, 2, 9, 1, 3};
    int n = 5;

    bubbleSort(arr, n);
```

```
    cout << "Sorted array: ";
    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

**Code:**

```python
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n - 1):
        # Last i elements are already in place
        for j in range(n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater
            # than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

# Sample Test Case 1
arr1 = [5, 2, 9, 1, 3]
print(f"Original array 1: {arr1}")
bubble_sort(arr1)
print(f"Sorted array 1: {arr1}")

# Sample Test Case 2
arr2 = [64, 34, 25, 12, 22, 11, 90]
print(f"\nOriginal array 2: {arr2}")
bubble_sort(arr2)
print(f"Sorted array 2: {arr2}")

# Sample Test Case 3 (already sorted)
arr3 = [1, 2, 3, 4, 5]
print(f"\nOriginal array 3: {arr3}")
bubble_sort(arr3)
print(f"Sorted array 3: {arr3}")
```

```python
# Sample Test Case 1
arr1 = [5, 2, 9, 1, 3]
print(f"Original array 1: {arr1}")
bubble_sort(arr1)
print(f"Sorted array 1: {arr1}")

# Sample Test Case 2
arr2 = [64, 34, 25, 12, 22, 11, 90]
print(f"\nOriginal array 2: {arr2}")
bubble_sort(arr2)
print(f"Sorted array 2: {arr2}")

# Sample Test Case 3 (already sorted)
arr3 = [1, 2, 3, 4, 5]
print(f"\nOriginal array 3: {arr3}")
bubble_sort(arr3)
print(f"Sorted array 3: {arr3}")

# Sample Test Case 4 (reverse sorted)
arr4 = [5, 4, 3, 2, 1]
print(f"\nOriginal array 4: {arr4}")
bubble_sort(arr4)
print(f"Sorted array 4: {arr4}")
```

**Output:**

```python
bubble_sort(arr4)
print(f"Sorted array 4: {arr4}")
```

```
Original array 1: [5, 2, 9, 1, 3]
Sorted array 1: [1, 2, 3, 5, 9]

Original array 2: [64, 34, 25, 12, 22, 11, 90]
Sorted array 2: [11, 12, 22, 25, 34, 64, 90]

Original array 3: [1, 2, 3, 4, 5]
Sorted array 3: [1, 2, 3, 4, 5]

Original array 4: [5, 4, 3, 2, 1]
Sorted array 4: [1, 2, 3, 4, 5]
```

**Explanation:**

• The C++ program implements the Bubble Sort algorithm to arrange numbers in ascending order.
• It repeatedly compares adjacent elements and swaps them if they are in the wrong order.
• Multiple passes are performed until the largest unsorted elements move to their correct positions.
• The prompt asks AI to convert this logic into Python while keeping the same sorting behavior.
• The final Python program should produce the same sorted output when tested with different inputs.

**Task-03:**

**Prompt:**

Convert the following SQL query into an equivalent Pandas DataFrame operation.

Requirements:

Use groupby() and aggregation functions.

Produce the same output as the SQL query.

Provide runnable Python Pandas code with sample data.

SQL Query:

```
SELECT department, SUM(sales) AS total_sales

FROM sales

GROUP BY department;
```

## Code:

```python
import pandas as pd

# Create a sample DataFrame to mimic the 'sales' table
data = {
    'department': ['Electronics', 'Clothing', 'Electronics', 'Books', 'Clothing', 'Electronics', 'Books'],
    'sales': [1000, 500, 1500, 300, 700, 2000, 450]
}
sales_df = pd.DataFrame(data)

print("Original DataFrame:")
display(sales_df)

# Perform the equivalent Pandas operation:
# SELECT department, SUM(sales) AS total_sales FROM sales GROUP BY department;
total_sales_by_department = sales_df.groupby('department')['sales'].sum().reset_index()

# Rename the 'sales' column to 'total_sales' to match the SQL alias
total_sales_by_department.rename(columns={'sales': 'total_sales'}, inplace=True)

print("\nTotal sales by department (equivalent to SQL query output):")
display(total_sales_by_department)
```

## Output:

```
Original DataFrame:
```

| | department | sales |
|---|---|---|
| 0 | Electronics | 1000 |
| 1 | Clothing | 500 |
| 2 | Electronics | 1500 |
| 3 | Books | 300 |
| 4 | Clothing | 700 |
| 5 | Electronics | 2000 |
| 6 | Books | 450 |

```
Total sales by department (equivalent to SQL query output):
```

| | department | total_sales |
|---|---|---|
| 0 | Books | 750 |
| 1 | Clothing | 1200 |
| 2 | Electronics | 4500 |

**Explanation:**

• The SQL query groups the records based on the department column and calculates the total sales for each department using the SUM() aggregate function.
• In Pandas, the same operation is performed using groupby() to group rows by department.
• The sum() function aggregates the sales values within each group just like SQL aggregation.
• The result is converted back into a normal table using reset_index() so it looks similar to SQL output.
• Both SQL and Pandas produce identical results showing total sales calculated for every department.

**Task-04:**

**Prompt:**

Convert the following Python Flask REST API into an equivalent Java Spring Boot application. Requirements: - Maintain the same API endpoints and functionality. - Ensure identical request and response behavior. - Use proper framework conventions. - Provide complete executable Java code. Python Code:

```
# app.py

from flask import Flask, jsonify, request


app = Flask(__name__)


students = []


@app.route("/students", methods=["GET"])

def get_students():

    return jsonify(students)
```

```
@app.route("/students", methods=["POST"])
def add_student():
    data = request.json
    students.append(data)
    return jsonify({"message": "Student added"}), 201
if __name__ == "__main__":
    app.run(debug=True)
```

**Code:**

```
%%bash
mkdir -p src/main/java/com/example/demo
mkdir -p src/main/resources
```

```
%%writefile pom.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instanc
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.5</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.example</groupId>
    <artifactId>demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>demo</name>
    <description>Demo project for Spring Boot</description>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
```

```
%%writefile src/main/java/com/example/demo/Student.java
package com.example.demo;

public class Student {
    private Long id;
    private String name;
    private int age;

    public Student() {
    }

    public Student(Long id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }
```

```
%%writefile src/main/java/com/example/demo/StudentController.java
package com.example.demo;

import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.atomic.AtomicLong;

@RestController
@RequestMapping("/students")
public class StudentController {

    private final List<Student> students = new ArrayList<>();
    private final AtomicLong counter = new AtomicLong();

    public StudentController() {
        // Initialize with some dummy data
        students.add(new Student(counter.incrementAndGet(), "Alice", 20));
        students.add(new Student(counter.incrementAndGet(), "Bob", 22));
    }

    @GetMapping
    public List<Student> getAllStudents() {
        return students;
    }
```

```
    @GetMapping("/{id}")
    public Optional<Student> getStudentById(@PathVariable Long id) {
        return students.stream().filter(student -> student.getId().equals(id)).findFirst();
    }

    @PostMapping
    public Student addStudent(@RequestBody Student newStudent) {
        newStudent.setId(counter.incrementAndGet());
        students.add(newStudent);
        return newStudent;
    }

    @DeleteMapping("/{id}")
    public void deleteStudent(@PathVariable Long id) {
        students.removeIf(student -> student.getId().equals(id));
    }
}
```

```
Writing src/main/java/com/example/demo/StudentController.java
```

inally, here is the main `SpringBootApplication` class. The `@SpringBootApplication` annotation mark
s a Spring Boot application, and the `main` method uses `SpringApplication.run()` to start the embed
omcat server and deploy our application.

```
%%writefile src/main/java/com/example/demo/DemoApplication.java
package com.example.demo;
```

# Output:

**1. Get all students:**

```
%%bash
curl http://localhost:8080/students
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
```

**2. Get student by ID (e.g., ID 1):**

```
%%bash
curl http://localhost:8080/students/1
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
```

**3. Add a new student:**

```
%%bash
curl -X POST -H "Content-Type: application/json" -d '{"name":"Charlie","age":21}' http://localhost:8086
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100    27    0     0  100    27      0  12581 --:--:-- --:--:-- --:--:-- 27000
```

**4. Get all students again to see the newly added student:**

```
%%bash
curl http://localhost:8080/students
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
```

**5. Delete a student (e.g., student with ID 2):**

```
%%bash
curl -X DELETE http://localhost:8080/students/2
```

**5. Delete a student (e.g., student with ID 2):**

```
[38]
✓ 0s
%%bash
curl -X DELETE http://localhost:8080/students/2
```

```
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
```

**6. Get all students again to confirm deletion:**

```
[39]
✓ 0s
%%bash
curl http://localhost:8080/students
```

```
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
```

**Explanation:**

• The Java program creates a Spring Boot application that runs a REST API similar to the Python Flask server.
• DemoApplication.java starts the Spring Boot server and initializes the application.
• StudentController.java defines API endpoints using annotations like @GetMapping and @PostMapping.
• A list is used as in-memory storage to save student data, similar to the Python list.
• The GET request returns all students, while the POST request adds a new student and sends a JSON confirmation response.

**Task-05:**

**Prompt:**

Convert the following Python program into an equivalent C++ program.

Requirements:

- Maintain the same logic for checking even or odd numbers.

- Use proper C++ syntax and input/output statements.

- Ensure identical output formatting.
- Provide complete runnable C++ code.
- Allow user input and test with multiple values.

Python Code:

```
def check_even_odd(num):
    if num % 2 == 0:
        print("Even")
    else:
        print("Odd")

n = int(input("Enter a number: "))
check_even_odd(n)
```

**Code:**

```
%%writefile even_odd.cpp
#include <iostream>

// Function to check if a number is even or odd
void checkEvenOdd(int num) {
    if (num % 2 == 0) {
        std::cout << "Even" << std::endl;
    } else {
        std::cout << "Odd" << std::endl;
    }
}

int main() {
    int n;
    std::cout << "Enter a number: ";
    std::cin >> n;
    checkEvenOdd(n);

    // Test with multiple values as requested
    std::cout << "\nTesting with other values:\n";
    std::cout << "Enter another number: ";
    std::cin >> n;
    checkEvenOdd(n);

    std::cout << "Enter a final number: ";
    std::cin >> n;
    checkEvenOdd(n);

    return 0;
```

```
%%bash
g++ even_odd.cpp -o even_odd
```

Finally, let's run the compiled C++ program and test it with a

Gemini

```
%%bash

echo "Testing with input: 10"
echo "10" | ./even_odd

echo "\nTesting with input: 7"
echo "7" | ./even_odd

echo "\nTesting with input: -4"
echo "-4" | ./even_odd
```

**Output:**

```
Testing with input: 10
Enter a number: Even
\nTesting with input: 7
Enter a number: Odd
\nTesting with input: -4
Enter a number: Even
```

**Explanation:**

The Python program defines a function that checks whether a number is even or odd using the modulus operator (%).

It takes a number as input from the user and calls the function to perform the check.

If the number is divisible by 2, it prints “Even”; otherwise, it prints “Odd.”
The prompt asks AI to convert this same logic into C++ while keeping input, output, and behavior unchanged.
The C++ program uses cin and cout for input/output and produces the same result as the Python version for all test values.